Breaking the Algorithm: A Source Code Analysis of X's 2026 Recommendation System and Its Implications for Small Account Growth

Boris Djordjevic

199 Biotechnologies · March 2026

Abstract. We present a technical analysis of X's current recommendation algorithm based on the open-source release of `xai-org/x-algorithm` (January 2026). The system replaces the 2023 `twitter/the-algorithm` entirely, eliminating TweepCred reputation scoring, SimClusters community detection, and Real Graph relationship modelling in favour of a Grok-based transformer that learns exclusively from user engagement sequences. We examine the complete scoring pipeline —19 engagement signals combined via a weighted scorer with asymmetric negative compression— and derive practical implications for accounts with fewer than 100 followers attempting to grow from a dormant state. Every claim is tagged by evidence source: source code (**CODE**), published empirical research (**EMP.**), or first-principles inference (**INF.**). Weight constants, excluded from the open-source release, are estimated through triangulation of code structure, 2023 baselines, and community experiments.

---

1 Introduction

On January 20, 2026, X (formerly Twitter) released the source code of its recommendation algorithm under the Apache 2.0 licence at `github.com/xai-org/x-algorithm`. This is a complete rewrite of the 2023 open-source release (`twitter/the-algorithm`), replacing the Scala/Java stack with Rust for the serving layer and Python/JAX for the machine learning components.

The 2023 release produced a cottage industry of growth advice based on specific weight constants (e.g., "replies are weighted 13.5×, reports −369×"). Much of this advice persists in 2026, despite the underlying systems having been entirely replaced. The 2026 README states plainly: *"We have eliminated every single hand-engineered feature"* **CODE.**

This paper analyses the 2026 source code to determine: (a) what signals the algorithm actually uses, (b) how the scoring pipeline works, (c) what can be deduced about relative signal importance, and (d) what this means for a specific use case—a dormant account with approximately 100 followers attempting to grow in a niche topic area.

1.1 Scope and methodology

We analyse every `.rs` and `.py` file in the repository. Where weight constants are referenced but not published (the `params` module is excluded "for security reasons"), we estimate relative magnitudes using three evidence sources, each tagged throughout:

- **CODE** Direct observation from published source code.
- **EMP.** Derived from published 2025–2026 empirical research, including Buffer's study of 18.8M posts and community experiments.
- **INF.** First-principles reasoning from code architecture and platform economics.

1.2 Superseded systems

The following 2023 systems are absent from the 2026 codebase and should no longer be cited:

2023 System	2026 Status
TweepCred (PageRank reputation, 0–100)	Eliminated
SimClusters (145K community embeddings)	Replaced by Phoenix two-tower retrieval
Real Graph (interaction edge weights)	Replaced by engagement sequence learning
MaskNet Heavy Ranker (48M params)	Replaced by Grok transformer
EarlyBird Light Ranker	Eliminated
`reply_engaged_by_author` signal (+75.0)	Removed
Bookmark signal	Not present in 2026 scorer
Blue Verified in-network 4× / OON 2×	Not present in 2026 recommendation code

Table 1: Systems present in the 2023 open-source release that are absent from the 2026 codebase.

2 System Architecture

The system comprises four components: Home Mixer (orchestration), Thunder (in-network post store), Phoenix (ML ranking and retrieval), and a composable Candidate Pipeline framework.

2.1 Pipeline stages

A request traverses nine stages in sequence CODE:

1. **Query Hydration** — `UserActionSeqQueryHydrator` fetches the user's last 128 engagements; `UserFeaturesQueryHydrator` fetches following/blocked/muted lists.
2. **Candidate Sourcing** — Thunder provides in-network posts; Phoenix Retrieval provides out-of-network posts via two-tower ANN search.
3. **Candidate Hydration** — Core data, author info, video duration, subscription status enriched in parallel.
4. **Pre-Scoring Filters** — 10 sequential filters remove duplicates, old posts, self-posts, blocked/muted authors, and previously seen content.
5. **Phoenix Scorer** — Grok transformer predicts $P(\text{action})$ for 19 engagement types.
6. **Weighted Scorer** — Combines predictions into a single score via weighted sum with asymmetric offset.
7. **Author Diversity Scorer** — Applies exponential decay to repeated authors.
8. **OON Scorer** — Applies multiplicative discount to out-of-network candidates.
9. **Selection and Post-Filtering** — Top-K selection, then visibility filtering (safety) and conversation deduplication.

2.2 Phoenix: The Grok Transformer

The ranking model (`recsys_model.py`) is a transformer adapted from Grok-1 with three key properties:

Hash-based embeddings. Users, posts, and authors are represented via multiple hash functions mapped to embedding tables, eliminating hand-crafted features CODE.

Engagement history as context. The model takes as input a sequence of $[\textit{User} \mid \textit{History}\ (S = 128) \mid \textit{Candidates}\ (C = 32)]$ embeddings. History entries encode the post, its author, the action taken, and the product surface CODE.

Candidate isolation masking. An attention mask prevents candidates from attending to each other while allowing them to attend to user context. This ensures scores are batch-independent and cacheable CODE.

2.3 Phoenix Retrieval: Two-Tower Model

Out-of-network discovery uses a two-tower architecture CODE:

- **User Tower**: Encodes engagement history via the same Grok transformer, producing an L2-normalised user embedding.
- **Candidate Tower**: Projects post+author embeddings through a two-layer MLP with SiLU activation, also L2-normalised.
- **Retrieval**: Dot product similarity between user and candidate embeddings, selecting top-K.

3 The 19 Engagement Signals

The `WeightedScorer` (`weighted_scorer.rs`) combines exactly 19 predicted engagement probabilities into a single score CODE. Weight constants reside in `params.rs`, which is excluded from the release.

3.1 Signal enumeration

	Signal	Action	Est. Wt.
1	Favorite (like)	`ServerTweetFav`	1×
2	Reply	`ServerTweetReply`	∼20×
3	Retweet (repost)	`ServerTweetRetweet`	∼3×
4	Photo expand	`ClientTweetPhotoExpand`	∼2×
5	Click (conversation)	`ClientTweetClick`	∼10×
6	Profile click	`ClientTweetClickProfile`	∼12×
7	Video quality view	`ClientTweetVideoQualityView`	∼3×
8	Share (generic)	`ClientTweetShare`	∼10×
9	Share via DM	`ClientTweetClickSendViaDirectMessage`	∼25×
10	Share via copy link	`ClientTweetShareViaCopyLink`	∼20×
11	Dwell (binary)	`ClientTweetRecapDwelled`	∼8×
12	Quote tweet	`ServerTweetQuote`	∼18×
13	Quoted click	`ClientQuotedTweetClick`	∼4×
14	Dwell time (continuous)	`DwellTime`	∼0.1/s
15	Follow author	`ClientTweetFollowAuthor`	∼30×
16	Not interested	`ClientTweetNotInterestedIn`	$\sim -20\times$
17	Block author	`ClientTweetBlockAuthor`	$\sim -74\times$
18	Mute author	`ClientTweetMuteAuthor`	$\sim -40\times$
19	Report	`ClientTweetReport`	$\sim -369\times$

Table 2: All 19 engagement signals from `weighted_scorer.rs`. Weights 1–15 are positive; 16–19 are negative. Estimated weights use favorite = 1× as baseline. Sources: CODE (signal existence), EMP. INF. (weight estimates).

3.2 Scoring formula

The combined score is computed as:

$$\text{score} = \sum_{i=1}^{19} w_i \cdot P(a_i)$$

where $P(a_i)$ is the predicted probability of action a_i from the Grok transformer (converted from log-probabilities via $\exp(\cdot)$), and w_i is the corresponding weight constant CODE.

3.3 Negative score compression

The `offset_score()` function applies asymmetric treatment CODE:

```
fn offset_score(combined_score: f64) -> f64 {
    if combined_score < 0.0 {
        (combined_score + NEGATIVE_WEIGHTS_SUM) / WEIGHTS_SUM
            * NEGATIVE_SCORES_OFFSET
    } else {
        combined_score + NEGATIVE_SCORES_OFFSET
    }
}
```

Positive scores scale linearly with an additive offset. Negative scores are compressed into a bounded band near zero. This creates an architectural asymmetry: even moderate negative predictions effectively suppress content, while positive signals stack without limit INF..

3.4 New signals absent in 2023

Three categories of signals are entirely new in the 2026 scorer:

Share signals (3). `share_score`, `share_via_dm_score`, and `share_via_copy_link_score` each receive independent weights. The 2023 system had no share tracking in the scorer. The architectural separation implies premium weighting—DM shares in particular represent the highest-conviction sharing action (personal vouching) INF..

Dwell signals (2). Binary dwell (`ClientTweetRecapDwelled`) captures whether the user paused; continuous dwell time (`DwellTime`) captures duration. The 2023 system had only an implicit dwell signal via click metrics CODE.

Follow author. `ClientTweetFollowAuthor` is a first-class scoring signal. Content that causes follows receives direct algorithmic reward CODE.

3.5 Video quality view gating

The VQV signal is gated behind a minimum duration threshold CODE:

```
fn vqv_weight_eligibility(candidate: &PostCandidate) -> f64 {
    if candidate.video_duration_ms
        .is_some_and(|ms| ms > p::MIN_VIDEO_DURATION_MS)
    {
        p::VQV_WEIGHT
    } else {
        0.0
    }
}
```

Videos shorter than `MIN_VIDEO_DURATION_MS` receive zero VQV contribution. The threshold value is not published.

4 Penalties and Negative Signals

The algorithm enforces penalties at two layers: scoring (probabilistic, continuous) and filtering (binary, absolute).

4.1 Predictive penalties

The four negative signals are *predictions*, not events **CODE**. The Grok transformer predicts the probability that a user *would* block, mute, report, or dismiss a post—and penalises accordingly *before the user acts*. The model learns from historical negative actions to generalise across audience segments.

This means an account that has accumulated blocks or mutes trains the model to predict higher $P(\text{block})$ and $P(\text{mute})$ for that account's future content across all users **INF.**.

4.2 Hard filters

Ten pre-scoring filters remove content entirely **CODE**:

Filter	Effect
`DropDuplicatesFilter`	Remove duplicate post IDs
`CoreDataHydrationFilter`	Remove posts that failed metadata hydration
`AgeFilter`	Remove posts older than `MAX_POST_AGE` (Snowflake timestamp)
`SelfTweetFilter`	Remove the viewer's own posts
`RetweetDeduplicationFilter`	Deduplicate reposts of the same content
`IneligibleSubscriptionFilter`	Remove paywalled content user cannot access
`PreviouslySeenPostsFilter`	Remove posts already seen
`PreviouslyServedPostsFilter`	Remove posts already served this session
`MutedKeywordFilter`	Remove posts matching muted keywords (tokenised)
`AuthorSocialgraphFilter`	Remove posts from blocked/muted authors

Table 3: Pre-scoring filters. A post must survive all 10 to reach the scorer.

Post-selection, two additional filters apply: `VFFilter` (visibility filtering for safety—spam, violence, policy violations) and `DedupConversationFilter` **CODE**. Out-of-network content faces a stricter safety level (`TimelineHomeRecommendations` vs. `TimelineHome`).

4.3 Author diversity decay

The `AuthorDiversityScorer` applies exponential decay to repeated appearances by the same author **CODE**:

$$multiplier(n) = (1 - floor) \cdot decay^n + floor$$

where n is the zero-indexed position of this author's n-th post in the ranked feed. The first post receives a multiplier near 1.0; subsequent posts decay geometrically.

4.4 Out-of-network discount

The `OONScorer` multiplies out-of-network candidates by `OON_WEIGHT_FACTOR` (value hidden, confirmed < 1.0 by the code comment: *"Prioritize in-network candidates over out-of-network candidates"*) **CODE.**

5 Cold Start Analysis: The Dormant Small Account

We consider a specific profile: 96 followers, 104 following, account created 2023, dormant for an extended period, now reactivating. Free tier.

5.1 The empty history problem

The Grok transformer requires engagement history as input. The `UserActionSeqQueryHydrator` fetches recent engagements and, critically **CODE:**

```
if thrift_user_actions.is_empty() {
    return Err(format!("No user actions found for user {}", user_id));
}
```

An empty engagement history causes the entire scoring pipeline to short-circuit. The Phoenix Scorer returns candidates unscored; the Weighted Scorer produces zeros. The account is algorithmically invisible.

5.2 Retrieval model implications

The two-tower retrieval model builds a user embedding by encoding engagement history through the transformer and average-pooling **CODE.** With no history, the user vector degenerates to a generic/default embedding, producing undifferentiated similarity scores against the corpus. Out-of-network discovery is effectively disabled.

5.3 In-network as the only channel

Thunder serves posts from followed accounts as in-network candidates with sub-millisecond lookups **CODE.** This is the only reliable channel for a dormant account. However, in-network reach is bounded by the follower count (96), and the OON discount further penalises the only growth path.

5.4 Comparison with 2023

The 2023 system imposed explicit penalties on small accounts: TweepCred < 65 limited distribution to 3 tweets; the following/follower ratio penalty divided PageRank by $\exp(5 \cdot (r - 0.6))$. The 2026 system has no such explicit penalties. The discrimination is *behavioural*—absence of engagement data, not a hard threshold **INF.**.

This is both better and worse. Better: there is no follower-count floor to clear. Worse: the model cannot work *at all* without engagement data, whereas the 2023 system at least scored your 3 tweets.

6 Growth Strategy Derived from Source Code

6.1 Phase 1: Populate the history buffer (Days 1–14)

The immediate priority is generating engagement history to populate the 128-position sequence **CODE.** Without this, the transformer cannot function.

Daily actions:

- **Like 20–30 niche posts.** Each like enters `history_actions` and teaches the retrieval model your topic interests CODE.
- **Reply to 5–10 posts** from accounts with 1K–50K followers. Replies fire `ServerTweetReply`, estimated at ∼20× baseline weight. Target posts under 30 minutes old EMP..
- **Quote 2–3 posts.** Quote tweets fire `ServerTweetQuote` (separate from `ServerTweetRetweet`), estimated at ∼18× CODE.
- **Post 1–2 original tweets.** Text-only posts outperform video by 30% on X EMP..
- **DM 1–2 posts** to people who would value them. This fires `ClientTweetClickSendViaDirectMessage`—a separate, high-value signal CODE.

6.2 Phase 2: Consistent content creation (Days 15–60)

With engagement history populated, the transformer can score content. Increase original output:

- **3–5 posts per day**, spaced $\geq$ 2 hours apart to avoid the author diversity decay CODE.
- **1 thread per week** (5–7 tweets). Threads maximise the continuous `dwell_time` signal CODE.
- **1 image post per day** designed for tap-to-expand, triggering `photo_expand_score` CODE.
- **Subscribe to Premium at Day 21–30**. Buffer's study reports ∼10× reach for Premium accounts EMP.. The boost is multiplicative—subscribe only after building a base score to multiply INF..

6.3 Phase 3: Compounding growth (Days 60–180)

- Increase to 5–7 posts per day if quality is maintained.
- Add 1–2 video posts per week exceeding `MIN_VIDEO_DURATION_MS` CODE.
- Seek quote-post opportunities on larger accounts. `quoted_click_score` fires when users click through CODE.
- Share valuable posts via DM consistently—the most underrated signal in the system INF..

6.4 Content format selection

Format	Rationale	Evidence
Text	Highest engagement rate on X (0.48% vs. 0.41% for images/video). No production overhead enables frequency.	EMP.
Images	Triggers `photo_expand_score`. Design for forced expansion (small text, charts).	CODE
Threads	Maximises continuous `dwell_time` and binary `dwell_score`.	CODE
Video (> min. duration)	Qualifies for `vqv_score`. Below threshold = zero contribution.	CODE
Polls	No direct signal, but indirectly boosts `click_score` and `dwell_time`.	INF.

Table 4: Content format selection based on scoring signals.

6.5 Behaviours to avoid

Behaviour	Mechanism	Evidence
Getting reported	$P(\text{report}) \times \sim -369$ weight; `offset_score()` compression	CODE INF.
Getting blocked	$P(\text{block}) \times \sim -74$; trains model against future content	CODE INF.
Off-topic posting	Elevates $P(\text{not interested})$; content mismatch signal	INF.
Posting 5+ in 1 hour	Author diversity decay: decay^4 on 5th post	CODE
>100 likes/hour	Rate-limit shadowban (48–72 hours)	EMP.
Follow/unfollow cycling	3-month visibility reduction	EMP.
Combative tone	Grok predicts higher $P(\text{block})$, $P(\text{mute})$ even with engagement	EMP. INF.

Table 5: Behaviours that trigger negative signals or penalties.

7 Premium Subscription Analysis

The Premium/verification boost is *not present* in the 2026 recommendation source code—it is not among the 19 signals in `weighted_scorer.rs` CODE. However, empirical data consistently reports substantial reach advantages EMP.:

- Buffer (18.8M posts): Premium accounts average ∼600 impressions/post vs. significantly fewer for free accounts (∼10× advantage).
- Premium+ accounts average ∼1,550 impressions/post.
- Non-Premium accounts posting links showed zero median engagement from March–October 2025 (link penalty largely removed October 2025).

The boost likely operates at a different layer of the stack (delivery, CDN, or a pre-recommendation scoring step not included in the open-source release) INF..

Recommendation: Do not subscribe immediately. Build engagement history (2–3 weeks) first. A multiplicative boost on zero engagement produces zero. Subscribe at Day 21–30 when consistent posting yields measurable engagement INF..

8 Expected Milestones

Milestone	Timeline	Significance
Engagement history populated	Day 7–14	Phoenix transformer becomes functional
For You feed shows niche content	Day 7–10	Retrieval model learned embedding
First reply-back from target	Day 2–3	Engagement edge established
Premium subscription	Day 21–30	∼10× reach amplification
250 followers	Week 3–4	Meaningful in-network base
Post exceeding 1K impressions	Week 3–4	Out-of-network retrieval functioning
500 followers	Month 2–3	Compounding growth begins
First viral post (>10K)	Month 2–3	Embedding neighbourhood established
1,000 followers	Month 4–6	Self-sustaining growth flywheel

Table 6: Expected growth milestones for a dormant account reactivating with the described strategy.

9 Conclusion

The 2026 X recommendation algorithm represents a fundamental architectural shift from the 2023 system. The elimination of all hand-engineered features in favour of a Grok-based transformer means the algorithm is simultaneously more opaque (no published weight constants) and more elegant (a single model learns everything from engagement sequences).

For small dormant accounts, the primary challenge is the cold start problem—a transformer with no engagement history cannot score content. The solution is not "growth hacking" but rather systematic engagement to populate the 128-position history buffer, followed by consistent content creation that optimises for high-value signals (replies, shares, quotes, follows) while avoiding negative predictions (reports, blocks, mutes).

The most actionable findings from the source code:

1. **DM shares and copy-link shares are separate high-value signals**, brand new in 2026 and likely underweighted in current growth advice.
2. **The `reply_engaged_by_author` signal is removed**, eliminating the 150× "cheat code" of the 2023 system.
3. **Bookmarks are not a scoring signal**, contradicting widely circulated advice.
4. **Negative signals are predictive**, not reactive—the model penalises content it expects users to dislike before they act.
5. **The offset compression function** creates architectural asymmetry favouring punishment over reward, making reputation management critical.

9 References

1. xAI Corp. *x-algorithm*. GitHub, Apache 2.0, January 20, 2026. `github.com/xai-org/x-algorithm`
2. Twitter Inc. *the-algorithm*. GitHub, March 31, 2023. `github.com/twitter/the-algorithm`
3. Twitter Inc. *the-algorithm-ml*. GitHub, March 31, 2023. `github.com/twitter/the-algorithm-ml`
4. Buffer Research. "Does X Premium Really Boost Your Reach? We Analyzed 18.8 Million Posts." 2025. `buffer.com/resources/x-premium-review/`
5. TechCrunch. "X open sources its algorithm while facing a transparency fine." January 2026.

6. PostEverywhere. “How the X/Twitter Algorithm Works in 2026 (From the Source Code).” 2026. `posteverywhere.ai/blog/how-the-x-twitter-algorithm-works`
7. Tweet Archivist. “Complete Technical Breakdown: How the X Algorithm Works.” 2025–2026. `tweetarchivist.com/how-twitter-algorithm-works-2025`
8. Social Media Today. “X Reveals Key Signals for Post Reach.” 2025.
9. Pixelscan. “Twitter Shadowban: Causes, Detection & Fixes (2026 Guide).” `pixelscan.net/blog/twitter-shadowban-2026-guide/`
10. Circleboom. “The Hidden X Algorithm: TweepCred, Shadow Hierarchy, and Dwell Time.” 2025.
11. Tomorrow’s Publisher. “X Softens Stance on External Links.” October 2025.